Informe: control de congestión

Héctor Bonilla

Lázaro Narváez

Preguntas previas

¿Por qué usar Stop & Wait como base para Go-Back-N? Ambos usan secuencias, ACKs, timeouts y retransmisiones. Go-Back-N extiende esa lógica con una ventana de varios segmentos pendientes y ACKs acumulativos.

¿Cambia recv al usar SocketUDP y SlidingWindowCC en Stop & Wait? Su lógica se mantiene: recibir el segmento esperado, entregar sus datos una sola vez y confirmar duplicados. Las clases facilitan los timers y la ventana del emisor; solo pueden requerir adaptar las llamadas al socket.

¿Cuál es la ventana inicial con control? Un segmento, porque `cwnd = MSS = 8 bytes`. Sin control se mantiene una ventana de 8 segmentos.

Pruebas y comportamiento observado

Se ejecutaron las pruebas en Ubuntu/WSL2 con Python 3.12.3. Pasaron las 25 comprobaciones docentes con MSS de 1, 4, 7, 8 y 16 bytes. También funcionaron las lecturas parciales, el mensaje vacío y la recuperación al perder el último ACK. Los 22 experimentos de transferencia entregaron archivos íntegros y cerraron correctamente las conexiones.

Sin pérdidas, el archivo de 2048 B tardó 0,162 s con control y 0,025 s sin control. Ambos intercambiaron 257 segmentos de datos y 257 ACKs. Con control no hubo timeouts espontáneos: la ventana creció en slow start y `ssthresh` permaneció en `None`.

Con pérdidas, cada timeout redujo `ssthresh` a la mitad de la `cwnd` anterior y reinició `cwnd` a 8 bytes. Al alcanzar el umbral se pasó a congestion avoidance, como muestra el debug.

Comparación con 20% de pérdida

Se probaron 100 B y 2048 B porque el enunciado menciona ambos tamaños. Se hicieron cinco repeticiones por modo, con MSS de 8 bytes, timeout de 0,5 s y pérdida simulada del 20% en datos y ACKs, excluyendo handshake y cierre.

Se midió el tiempo de `send`, incluyendo retransmisiones y debug. Los mensajes cuentan intentos de envío de datos y ACKs, incluidos los descartados y retransmitidos.

Archivo	Control de congestión	Tiempo promedio (s)	Mensajes promedio
100 B	Sí	3.082	47.6
100 B	No	2.134	68.4
2048 B	Sí	42.052	790.6
2048 B	No	29.137	1267.2

Sin control se transmitió más rápido; con control se enviaron menos mensajes. La reducción de ventana tras los timeouts limita las retransmisiones, pero aumenta el tiempo de envío. Estos resultados corresponden a localhost sin congestión real ni retardo artificial.

Para 2048 B, el control redujo los mensajes un **38%**, pero aumentó el tiempo un **44%**.

Evidencia: resultados Linux (resultados_pruebas_linux.csv) y trazas de debug (debug_pruebas_linux.txt).